

详细设计说明书

项目名称：在线论文文档自动生成工具

版本：V2026.07

成员：	<u>42411051 李思贤</u>
成员：	<u>42411084 余锦标</u>
成员：	<u>42411094 谢宏涛</u>
成员：	<u>42411116 李佳蔚</u>
成员：	<u>42411134 张宗元</u>
成员：	<u>42411202 张熙伟</u>

实训课程软件工程文档

2026 年 7 月 15 日

目录

目录	1
1 数据库详细设计	3
1.1 数据库设计概述	3
1.2 学院表 (sys_college)	3
1.3 用户表 (sys_user)	3
1.4 论文模板表 (template)	4
1.5 模板版本表 (template_version)	5
1.6 论文主表 (thesis)	6
1.7 论文章节表 (thesis_section)	6
1.8 参考文献表 (thesis_reference)	7
1.9 图片资源表 (thesis_image)	8
1.10 提交记录表 (submission)	8
1.11 批注表 (annotation)	8
1.12 批阅记录表 (review_record)	9
1.13 章节草稿历史表 (thesis_section_draft_history)	9
1.14 索引设计	10
1.15 数据库初始化策略	10
2 认证与权限模块详细设计	10
2.1 模块概述	10
2.2 认证流程设计	11
2.2.1 用户注册流程	11
2.2.2 用户登录流程	11
2.2.3 用户登出流程	12
2.3 JWT Token 设计	12
2.3.1 Token 格式与内容	12
2.3.2 Token 签名算法	12
2.3.3 Token 解析与校验	13
2.4 请求认证过滤器 (JwtFilter)	13
2.5 角色权限拦截器 (RoleInterceptor)	14
2.6 @RoleRequired 注解	15
2.7 拦截器注册配置 (WebConfig)	16
2.8 密码加密方案	16
2.9 Redis Token 缓存策略	17
2.10 统一返回体设计	17
2.11 全局异常处理	18
2.12 安全防护措施	18
2.13 认证鉴权全链路时序	19

版本修订记录

版本	日期	修改人	审核	修改说明
V1.0	2026-07-17	C	全组	初稿完成

1 数据库详细设计

1.1 数据库设计概述

系统数据库采用 PostgreSQL 16 关系型数据库，数据库名称为 `thesis_generator`。整体遵循第三范式（3NF）设计，以论文生命周期为主线，围绕用户角色管理、模板配置管理、论文内容编辑、提交批阅流程四大业务域组织数据表结构。以下给出全部数据表的完整 DDL（Data Definition Language）语句及详细字段说明。

1.2 学院表（`sys_college`）

学院表用于存储高校下设各学院的基础信息，是整个系统数据组织体系的最上层实体。用户在注册时选择所属学院，模板按学院维度进行分类管理。

Listing 1: `sys_college` 建表语句

```
1 CREATE TABLE IF NOT EXISTS sys_college (  
2     id BIGSERIAL PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     code VARCHAR(50) NOT NULL UNIQUE,  
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
6     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
7 );
```

- `id`: BIGSERIAL 自增主键，数据库自动分配唯一标识；
- `name`: 学院全称，VARCHAR(100)，不可为空；
- `code`: 学院编号（如 CS、EE、MATH），VARCHAR(50)，唯一约束防止重复；
- `created\_at` / `updated\_at`: 时间戳字段，记录创建和更新时间。

1.3 用户表（`sys_user`）

用户表是系统权限体系的核心，存储全部用户（管理员、学生、教师）的账号信息、加密密码、角色标识和学院归属。

Listing 2: `sys_user` 建表语句

```
1 CREATE TABLE IF NOT EXISTS sys_user (  
2     id BIGSERIAL PRIMARY KEY,  
3     username VARCHAR(50) NOT NULL UNIQUE,  
4     password VARCHAR(255) NOT NULL,  
5     real_name VARCHAR(50) NOT NULL,  
6     role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN', 'STUDENT', 'TEACHER')),  
7     college_id BIGINT REFERENCES sys_college(id),  
8     enabled BOOLEAN DEFAULT TRUE,
```

```
9      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
10      updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );
```

- **username:** 登录用户名, VARCHAR(50), 唯一约束, 用于登录认证;
- **password:** BCrypt 哈希加密后的密码密文, VARCHAR(255) 以容纳 60 字符的 BCrypt 哈希值;
- **real_name:** 用户真实姓名, 用于前端界面显示;
- **role:** 角色枚举, CHECK 约束限定为 ADMIN、STUDENT、TEACHER 三种取值;
- **college_id:** 外键引用 sys_college.id, 可为空 (管理员可不归属特定学院);
- **enabled:** 账号启用状态, 禁用后该账号无法登录。

角色权限体系设计如表 1 所示:

表 1: 角色权限矩阵

功能权限	ADMIN	STUDENT	TEACHER
用户注册		(仅 STUDENT/TEACHER)	(仅 STUDENT/TEACHER)
模板管理 (CRUD)		×	×
学院管理		×	×
创建/编辑论文	×		×
文档导出	×		×
提交论文	×		×
批注/评阅论文	×	×	
查看批阅结果	×		

1.4 论文模板表 (template)

模板表存储由管理员创建的论文模板基本信息, 每个模板可关联特定学院, 并设定论文类型。

Listing 3: template 建表语句

```
1 CREATE TABLE IF NOT EXISTS template (
2     id BIGSERIAL PRIMARY KEY,
3     name VARCHAR(200) NOT NULL,
4     type VARCHAR(20) NOT NULL CHECK (type IN ('GRADUATION', 'COURSE', 'PROJECT')),
5     college_id BIGINT REFERENCES sys_college(id),
6     enabled BOOLEAN DEFAULT TRUE,
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
8     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
```

- **name:** 模板名称（如“计算机学院本科毕业论文模板”），VARCHAR(200)；
- **type:** 论文类型，CHECK 约束限定三种取值：GRADUATION（毕业论文）、COURSE（课程论文）、PROJECT（项目论文）；
- **college_id:** 外键引用 sys_college.id，实现按学院的模板隔离；
- **enabled:** 启用状态，停用的模板不显示给学生创建论文。

1.5 模板版本表（template_version）

模板版本表是系统灵活性的关键设计。利用 PostgreSQL 特有的 JSONB 类型存储可变结构的配置数据，避免了为不同模板类型设计大量冗余字段的问题。

Listing 4: template_version 建表语句

```

1 CREATE TABLE IF NOT EXISTS template_version (
2     id BIGSERIAL PRIMARY KEY,
3     template_id BIGINT NOT NULL REFERENCES template(id) ON DELETE CASCADE,
4     version_number VARCHAR(20) NOT NULL DEFAULT '1.0',
5     is_current BOOLEAN DEFAULT TRUE,
6     cover_fields JSONB NOT NULL DEFAULT '[]',
7     chapter_structure JSONB NOT NULL DEFAULT '[]',
8     format_config JSONB NOT NULL DEFAULT '{}',
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );

```

JSONB 配置字段的设计与使用场景：

- **cover_fields**（JSON 数组）：定义封面需填写的字段列表和显示顺序。示例：

```

1 [{"key": "title", "label": "论文题目", "required": true},
2  {"key": "author", "label": "作者姓名", "required": true},
3  {"key": "college", "label": "学院", "required": true}]

```

- **chapter_structure**（JSON 数组）：定义论文的章节树结构。示例：

```

1 [{"key": "abstract", "title": "摘要", "type": "front"},
2  {"key": "ch1", "title": "绪论", "type": "chapter", "children": [
3      {"key": "ch1_1", "title": "研究背景"},
4      {"key": "ch1_2", "title": "研究意义"}]},
5  {"key": "ref", "title": "参考文献", "type": "back"}]

```

- **format_config**（JSON 对象）：定义排版格式参数。示例：

```

1 {"fontFamily": "SimSun", "fontSize": 12, "lineSpacing": 1.5,
2  "marginTop": 2.5, "marginBottom": 2.5,
3  "marginLeft": 3.0, "marginRight": 2.5, "pageSize": "A4"}

```

ON DELETE CASCADE 确保删除模板时其全部版本记录同步删除，维护数据一致性。

1.6 论文主表 (thesis)

论文主表记录每篇论文的元信息和生命周期状态。

Listing 5: thesis 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis (
2     id BIGSERIAL PRIMARY KEY,
3     student_id BIGINT NOT NULL REFERENCES sys_user(id),
4     template_version_id BIGINT REFERENCES template_version(id),
5     title VARCHAR(500) NOT NULL DEFAULT '未命名论文',
6     status VARCHAR(20) NOT NULL DEFAULT 'DRAFT'
7     CHECK (status IN ('DRAFT', 'COMPLETED', 'SUBMITTED',
8                     'REVIEWED', 'RETURNED', 'GENERATING')),
9     is_locked BOOLEAN DEFAULT FALSE,
10    college_id BIGINT REFERENCES sys_college(id),
11    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
12    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
13 );

```

论文状态机流转规则如下，状态转换关系如图 1 所示：

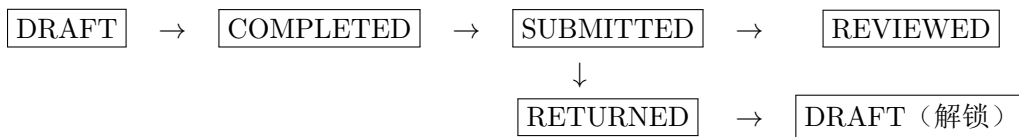


图 1: 论文状态流转图

- DRAFT → COMPLETED: 学生确认论文内容完成；
- COMPLETED → SUBMITTED: 学生提交至教师，is_locked 置为 TRUE，论文只读；
- SUBMITTED → REVIEWED: 教师完成批阅，给出分数和等级；
- SUBMITTED → RETURNED: 教师退回论文，附带退回原因，is_locked 置为 FALSE，学生可重新编辑；
- GENERATING: DOCX/PDF 导出任务进行中的中间状态。

1.7 论文章节表 (thesis_section)

章节表存储每篇论文的各章节结构信息和富文本 HTML 内容。

Listing 6: thesis_section 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_section (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     title VARCHAR(300) NOT NULL,

```

```

5      section_key VARCHAR(100) NOT NULL,
6      content TEXT DEFAULT '',
7      draft_content TEXT DEFAULT '',
8      section_type VARCHAR(50) DEFAULT 'chapter',
9      parent_id BIGINT REFERENCES thesis_section(id),
10     sort_order INT DEFAULT 0,
11     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
12     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
13 );

```

- `section\_key`: 章节的业务标识键（如 `ch1`、`ch1_1`、`abstract`），与模板 `version` 的 `chapter\_structure` 中的 `key` 字段对应；
- `content`: 正式保存的章节 HTML 内容；
- `draft\_content`: 自动保存的草稿内容，独立于正式保存，30 秒自动触发保存；
- `section\_type`: 章节类型标识（`chapter/section/subsection/front/back`），用于排版逻辑判断；
- `parent\_id`: 自引用外键，实现多级章节嵌套（父章节 → 子章节）；
- `sort\_order`: 同级章节的排序序号，整数递增，前端按此排序展示；
- `ON DELETE CASCADE`: 删除论文时自动删除全部章节记录。

1.8 参考文献表（`thesis\_reference`）

参考文献表遵循 GB/T 7714-2015 标准的数据元素设计。

Listing 7: `thesis\_reference` 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_reference (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     authors VARCHAR(500) NOT NULL,
5     title VARCHAR(500) NOT NULL,
6     journal VARCHAR(300),
7     year VARCHAR(10),
8     volume VARCHAR(50),
9     issue VARCHAR(50),
10    pages VARCHAR(50),
11    doi VARCHAR(200),
12    sort_order INT DEFAULT 0,
13    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14 );

```

字段设计覆盖了期刊论文、会议论文、学位论文、图书等常见文献类型的核心字段，`doi` 字段支持 DOI 链接跳转。

1.9 图片资源表 (thesis_image)

Listing 8: thesis_image 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_image (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     section_id BIGINT REFERENCES thesis_section(id),
5     original_name VARCHAR(500) NOT NULL,
6     stored_path VARCHAR(500) NOT NULL,
7     url VARCHAR(500) NOT NULL,
8     file_size BIGINT DEFAULT 0,
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );

```

- original_name: 用户上传时的原始文件名;
- stored_path: 服务端文件系统存储路径;
- url: 对外访问 URL (前端 img 标签 src 引用);
- file_size: 文件大小 (字节), 用于空间占用统计。

1.10 提交记录表 (submission)

Listing 9: submission 建表语句

```

1 CREATE TABLE IF NOT EXISTS submission (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     student_id BIGINT NOT NULL REFERENCES sys_user(id),
5     version_number INT NOT NULL DEFAULT 1,
6     submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );

```

每次学生提交论文时新增一条记录, version_number 递增, 保留完整的提交历史。

1.11 批注表 (annotation)

批注表记录教师在论文指定文字位置添加的修改建议。

Listing 10: annotation 建表语句

```

1 CREATE TABLE IF NOT EXISTS annotation (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     section_id BIGINT NOT NULL REFERENCES thesis_section(id),
5     teacher_id BIGINT NOT NULL REFERENCES sys_user(id),

```

```

6      start_offset INT NOT NULL,
7      text_length INT NOT NULL,
8      selected_text TEXT,
9      content TEXT NOT NULL,
10     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );

```

- `start\_offset`: 选中文字在原 HTML 内容中的起始字符位置 (0-based), 用于精确定位批注锚点;
- `text\_length`: 选中文字的长度, 结合 `start\_offset` 确定完整范围;
- `selected\_text`: 被选中的原文内容 (冗余存储, 便于离线查看批注含义);
- `content`: 教师的批注意见文本。

1.12 批阅记录表 (review_record)

Listing 11: review_record 建表语句

```

1 CREATE TABLE IF NOT EXISTS review_record (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     teacher_id BIGINT NOT NULL REFERENCES sys_user(id),
5     comment_html TEXT,
6     score INT CHECK (score >= 0 AND score <= 100),
7     grade VARCHAR(10),
8     action VARCHAR(20) NOT NULL CHECK (action IN ('REVIEWED', 'RETURNED')),
9     return_reason TEXT,
10    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );

```

分数 (`score`) 与等级 (`grade`) 的换算规则为: ≥ 90 分 $\rightarrow$ 优、80-89 $\rightarrow$ 良、70-79 $\rightarrow$ 中、60-69 $\rightarrow$ 及格、 < 60 $\rightarrow$ 不及格。`action` 区分批阅 (REVIEWED) 和退回 (RETURNED) 两种操作类型。

1.13 章节草稿历史表 (thesis_section_draft_history)

Listing 12: thesis_section_draft_history 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_section_draft_history (
2     id BIGSERIAL PRIMARY KEY,
3     section_id BIGINT NOT NULL REFERENCES thesis_section(id) ON DELETE CASCADE,
4     draft_content TEXT,
5     saved_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6 );

```

草稿历史表用于记录每次自动保存的快照, 支持学生回溯到先前的草稿版本。

1.14 索引设计

在概要设计中已介绍索引策略，以下给出索引的完整 DDL 语句：

Listing 13: 索引创建语句

```

1 CREATE INDEX IF NOT EXISTS idx_user_role ON sys_user(role);
2 CREATE INDEX IF NOT EXISTS idx_template_college ON template(college_id);
3 CREATE INDEX IF NOT EXISTS idx_template_type ON template(type);
4 CREATE INDEX IF NOT EXISTS idx_thesis_student ON thesis(student_id);
5 CREATE INDEX IF NOT EXISTS idx_thesis_status ON thesis(status);
6 CREATE INDEX IF NOT EXISTS idx_section_thesis ON thesis_section(thesis_id);
7 CREATE INDEX IF NOT EXISTS idx_annotation_thesis ON annotation(thesis_id);
8 CREATE INDEX IF NOT EXISTS idx_review_thesis ON review_record(thesis_id);

```

全部索引均采用 B-tree 默认结构。由于当前数据规模为实训项目级别（预计千级用户、万级论文记录），B-tree 索引足以满足所有查询场景的性能需求。若未来数据量增长至百万级，可针对 JSONB 字段（template_version 的配置数据）添加 GIN 索引以支持配置内容的高效检索。

1.15 数据库初始化策略

系统启动时通过 Spring Boot 的 SQL 初始化机制自动执行 schema.sql 脚本。在 application.properties 中配置如下：

Listing 14: SQL 初始化配置

```

1 spring.sql.init.mode=always
2 spring.sql.init.schema-locations=classpath:schema.sql
3 spring.jpa.hibernate.ddl-auto=validate

```

ddl-auto=validate 确保 JPA 实体定义与实际数据库表结构保持一致，若存在字段差异则启动报错，防止运行时出现隐式的数据不一致问题。

2 认证与权限模块详细设计

2.1 模块概述

认证与权限模块（M1）负责系统中全部用户的身份认证和操作授权。模块采用 JWT（JSON Web Token）无状态认证机制配合 Redis 缓存的自研轻量级方案，未引入 Spring Security 全套框架，以降低学习曲线和项目复杂度，同时满足实训项目的安全需求。

模块核心职责包括：

1. **用户认证**：处理注册（创建账号）、登录（签发 Token）、登出（销毁 Token）三项基本认证操作；
2. **Token 管理**：JWT Token 的生成、解析、过期校验，以及 Redis 中的缓存存储；

3. **请求拦截:** 在 API 请求处理管道中拦截所有非公开请求, 校验 Token 有效性并提取用户身份信息;
4. **角色鉴权:** 根据 API 接口声明的所需角色, 校验当前请求用户的角色是否在允许列表中。

2.2 认证流程设计

2.2.1 用户注册流程

用户注册的完整处理流程如下:

1. 前端收集用户名、密码、真实姓名、角色 (STUDENT 或 TEACHER)、所属学院 ID, 通过 POST 请求发送至 `/api/v1/auth/register`;
2. 请求体经 `@Valid` 注解触发 JSR-303 校验 (用户名 4-50 位、密码 6-100 位、必填字段非空);
3. `AuthController.register()` 接收 `RegisterRequest` DTO, 调用 `AuthService.register()`;
4. `AuthService` 检查用户名是否已存在 (`userRepository.existsByUsername`), 如已存在则抛出 `BusinessException(400, "用户名已存在")`;
5. 校验通过后, 构造 `User` 实体对象: 使用 `BCryptPasswordEncoder.encode()` 对明文密码进行哈希加密, 设置 `enabled=true`, 调用 `userRepository.save()` 持久化;
6. 为新用户生成 JWT Token (`jwtUtil.generateToken`), Token 中包含 `userId`、`username`、`role` 三个 Claim;
7. 将 Token 写入 Redis 缓存 (key 格式: `token:<userId>`), 设置 24 小时过期, 与 JWT 过期时间一致;
8. 返回 `LoginResponse` (`token`、`userId`、`username`、`realName`、`role`)。

2.2.2 用户登录流程

登录流程与注册类似但更简化:

1. 前端发送 POST 请求至 `/api/v1/auth/login`, 携带 `LoginRequest` (`username + password`);
2. `AuthService` 根据用户名查询用户 (`userRepository.findByUsername`), 找不到则抛出 `BusinessException(401)`;
3. 检查 `enabled` 状态, 若为 `false` 则抛出 `BusinessException(403, "账号已被禁用")`;
4. 使用 `BCryptPasswordEncoder.matches()` 方法比对明文密码与数据库中的密文;
5. 认证通过后生成 JWT Token 并写入 Redis, 返回 `LoginResponse`。

2.2.3 用户登出流程

1. 前端发送 POST 请求至 `/api/v1/auth/logout` (需携带 Token);
2. `AuthController` 从 Request Attribute 中提取 `userId` (由 `JwtFilter` 预先注入);
3. `AuthService` 删除 Redis 中对应的 Token 缓存;
4. Token 本身仍在其有效期内, 但由于后端不再认可该 Token (后续请求在 Redis 中找不到对应记录), 实现了逻辑登出。

2.3 JWT Token 设计

2.3.1 Token 格式与内容

系统采用 `JWT` 库 (`io.jsonwebtoken 0.12.6`) 实现 JWT Token 的签发和解析。Token 结构为标准的三段式 JWT 格式: `Header.Payload.Signature`。

Payload (Claims) 包含以下字段:

表 2: JWT Token Payload 字段定义

字段名	类型	说明
<code>userId</code>	<code>Long</code>	用户 ID, 主键值
<code>username</code>	<code>String</code>	登录用户名
<code>role</code>	<code>String</code>	角色标识 (ADMIN/STUDENT/TEACHER)
<code>iat</code>	<code>Date</code>	签发时间 (Issued At)
<code>exp</code>	<code>Date</code>	过期时间 (Expiration), <code>iat + 24 小时</code>

2.3.2 Token 签名算法

Token 签名采用 HMAC-SHA256 对称密钥算法, 密钥通过 `application.properties` 中的 `jwt.secret` 配置项注入。密钥长度建议不少于 256 位 (32 字节), 以确保足够的签名强度。当前开发环境的密钥为占位符, 生产部署前须替换为随机生成的强密钥。

Listing 15: `JwtUtil.generateToken()` 核心逻辑

```
1 public String generateToken(Long userId, String username, String role) {
2     Map<String, Object> claims = new HashMap<>();
3     claims.put("userId", userId);
4     claims.put("username", username);
5     claims.put("role", role);
6
7     return Jwts.builder()
8         .claims(claims)
9         .issuedAt(new Date())
10        .expiration(new Date(System.currentTimeMillis() + expiration))
```

```

11         .signWith(getKey())    // HMAC-SHA256 签名
12         .compact();
13     }

```

2.3.3 Token 解析与校验

Token 解析时，JJWT 库自动校验签名和过期时间。若签名不匹配、Token 已过期或格式错误，JwtUtil.parseToken() 抛出相应异常，由 JwtFilter 捕获并统一处理（认证失败时不阻塞请求，仅不注入用户信息，由后续 RoleInterceptor 按需拒绝）。

Listing 16: JwtUtil.parseToken() 核心逻辑

```

1 public Claims parseToken(String token) {
2     return Jwts.parser()
3         .verifyWith(getKey())
4         .build()
5         .parseSignedClaims(token)
6         .getPayload();
7 }

```

2.4 请求认证过滤器（JwtFilter）

JwtFilter 是认证机制的核心组件，继承自 Spring Web 的 OncePerRequestFilter 抽象类，确保每次请求仅执行一次过滤逻辑。

Listing 17: JwtFilter.doFilterInternal() 核心逻辑

```

1 @Override
2 protected void doFilterInternal(HttpServletRequest request,
3     HttpServletResponse response, FilterChain chain)
4     throws ServletException, IOException {
5
6     String path = request.getRequestURI();
7
8     // 放行公开端点
9     if (path.startsWith("/api/v1/auth/") ||
10         path.contains("/doc.html") ||
11         path.contains("/v3/api-docs") ||
12         path.contains("/swagger") ||
13         path.contains("/webjars")) {
14         chain.doFilter(request, response);
15         return;
16     }
17
18     // 解析Token并注入用户信息
19     String header = request.getHeader("Authorization");
20     if (header != null && header.startsWith("Bearer ")) {

```

```

21     String token = header.substring(7);
22     try {
23         if (!jwtUtil.isTokenExpired(token)) {
24             Claims claims = jwtUtil.parseToken(token);
25             request.setAttribute("userId",
26                 claims.get("userId", Long.class));
27             request.setAttribute("username",
28                 claims.get("username", String.class));
29             request.setAttribute("role",
30                 claims.get("role", String.class));
31         }
32     } catch (Exception ignored) {
33         // Token无效时不注入属性，后续鉴权拦截器按需拒绝
34     }
35 }
36
37 chain.doFilter(request, response);
38 }

```

关键设计决策说明：

- **不阻断未认证请求：**Filter 不在缺少 Token 或 Token 无效时直接返回 401，而是将请求继续传递给后续拦截器。这样设计的好处是：若后续 Controller 方法未标注 `@RoleRequired`，则允许未登录用户访问（为未来可能的公开接口预留灵活性）；
- **白名单路径：**注册、登录、API 文档（Swagger/Knife4j）相关路径被明确排除在认证拦截之外；
- **无状态设计：**Filter 不查询数据库，仅依赖 JWT Token 自包含的用户信息完成身份识别，减轻数据库压力。

2.5 角色权限拦截器（RoleInterceptor）

RoleInterceptor 实现 Spring MVC 的 `HandlerInterceptor` 接口，在 Controller 方法执行前校验用户角色。

Listing 18: RoleInterceptor.preHandle() 核心逻辑

```

1  @Override
2  public boolean preHandle(HttpServletRequest request,
3                          HttpServletResponse response, Object handler) {
4
5      if (!(handler instanceof HandlerMethod hm)) {
6          return true;
7      }
8
9      // 获取方法上的 @RoleRequired 注解
10     RoleRequired annotation = hm.getMethodAnnotation(RoleRequired.class);

```

```

11     if (annotation == null) {
12         annotation = hm.getBeanType().getAnnotation(RoleRequired.class);
13     }
14     if (annotation == null) {
15         return true; // 无注解则放行
16     }
17
18     String role = (String) request.getAttribute("role");
19     if (role == null) {
20         throw new BusinessException(401, "请先登录");
21     }
22
23     List<String> allowed = Arrays.asList(annotation.value());
24     if (!allowed.contains(role)) {
25         throw new BusinessException(403,
26             "权限不足, 需要角色: " + String.join(", ", allowed));
27     }
28
29     return true;
30 }

```

拦截器的工作流程如下:

1. 判断当前 handler 是否为 Controller 方法 (HandlerMethod), 非方法类型 (如静态资源请求) 直接放行;
2. 优先从方法上获取 @RoleRequired 注解, 若不存在则从 Controller 类上获取, 两级均无则放行 (无权限要求);
3. 从 Request Attribute 中取出 role 值 (由 JwtFilter 预先注入), 若为 null 则用户未登录, 抛出 BusinessException(401);
4. 校验当前用户角色是否在 @RoleRequired 声明的允许角色列表中, 不在则抛出 BusinessException(403);
5. 校验通过后返回 true, 请求继续进入 Controller 方法处理。

2.6 @RoleRequired 注解

自定义注解 @RoleRequired 用于在 Controller 类或方法级别声明接口所需的角色权限。

Listing 19: @RoleRequired 注解定义

```

1 @Target({ElementType.METHOD, ElementType.TYPE})
2 @Retention(RetentionPolicy.RUNTIME)
3 public @interface RoleRequired {
4     String[] value() default {};
5 }

```


典型使用方式:

- **类级别声明:** 在 Controller 类上标注 @RoleRequired, 该类下所有方法均需指定角色才能访问。例如管理员 Controller 标注@RoleRequired("ADMIN");
- **方法级别声明:** 在特定方法上标注, 覆盖类级别设置。例如在学生 Controller 中, 大部分方法需要 STUDENT 角色, 但某个查询接口允许 TEACHER 查看, 则在方法上标注@RoleRequired({"STUDENT", "TEACHER"});

2.7 拦截器注册配置 (WebConfig)

WebConfig 实现 WebMvcConfigurer 接口, 将 RoleInterceptor 注册到 Spring MVC 拦截器链中。

Listing 20: WebConfig 拦截器注册

```

1 @Configuration
2 @RequiredArgsConstructor
3 public class WebConfig implements WebMvcConfigurer {
4
5     private final RoleInterceptor roleInterceptor;
6
7     @Override
8     public void addInterceptors(InterceptorRegistry registry) {
9         registry.addInterceptor(roleInterceptor)
10            .addPathPatterns("/api/**")
11            .excludePathPatterns(
12                "/api/v1/auth/**", // 认证接口公开
13                "/doc.html",       // Knife4j 文档页
14                "/v3/api-docs/**", // OpenAPI JSON
15                "/swagger/**",    // Swagger 静态资源
16                "/webjars/**"     // Webjars 资源
17            );
18     }
19 }
```

关键配置说明:

- `addPathPatterns("/api/**")`: 拦截所有 API 请求;
- `excludePathPatterns(...)`: 排除认证接口和 API 文档相关路径, 这些路径在 JwtFilter 中同样被放行, 形成双重保障。

2.8 密码加密方案

系统采用 BCrypt 单向哈希算法存储用户密码。BCrypt 是专门为密码哈希设计的算法, 具有以下优势:

- **内置盐值:**每次加密自动生成随机盐值并嵌入哈希结果中(格式:\$2a\$10\$<22位盐值><31位哈希值>), 相同密码每次加密结果不同, 有效防御彩虹表攻击;
- **计算代价可调:**work factor 参数(示例为 10)表示 2^{10} 轮哈希迭代, 可根据硬件性能调整。当前默认值 10 在保证安全性的同时, 单次加密耗时约 100ms, 对登录体验影响可忽略;
- **不可逆:**BCrypt 是单向哈希, 无法从密文反推明文密码。

AuthService 中直接创建 BCryptPasswordEncoder 实例:

```
1 private final BCryptPasswordEncoder passwordEncoder =
2     new BCryptPasswordEncoder();
```

注册时调用 `passwordEncoder.encode(rawPassword)` 生成密文存储;登录时调用 `passwordEncoder.matches(r` 比对密码。

2.9 Redis Token 缓存策略

Token 在签发的同时写入 Redis 缓存, key 设计为 "token:" + `userId`, value 为 Token 字符串本身。设置过期时间为 24 小时, 与 JWT Token 的过期时间保持一致。

Redis Token 缓存的作用:

- **登出管理:**前端登出时, 后端删除 Redis 中的 Token 记录, 使得该 Token 不再被认可(即便 JWT 本身未过期);
- **单点登录控制:**可选扩展——同一 `userId` 的 Token 写入 Redis 时覆盖旧值, 天然实现了同一账号后续登录踢掉前一次登录的效果;
- **状态查询:**管理员可通过 Redis 中是否存在某用户的 Token 来判断其在线状态。

RedisTemplate 的序列化配置(RedisConfig)采用 String 序列化 key、GenericJackson2JsonRedisSerializer 序列化 value, 支持对象数据的 JSON 序列化存储, 同时保持 Redis 客户端中的 key 可读性。

2.10 统一返回体设计

所有 API 响应均通过 `Result<T>` 泛型类统一封装, 结构如下:

Listing 21: Result.java 统一返回体

```
1 @Data
2 @NoArgsConstructor
3 @AllArgsConstructor
4 public class Result<T> {
5     private int code;           // 业务状态码
6     private String message;     // 提示信息
7     private T data;             // 响应数据 (泛型)
8 }
```

```
9     public static <T> Result<T> ok(T data) {
10         return new Result<>(200, "success", data);
11     }
12
13     public static <T> Result<T> error(int code, String message) {
14         return new Result<>(code, message, null);
15     }
16 }
```

状态码约定：

- 200：请求成功；
- 400：参数校验失败；
- 401：未登录或 Token 无效；
- 403：角色权限不足；
- 500：服务端内部错误。

2.11 全局异常处理

GlobalExceptionHandler 通过 @RestControllerAdvice 注解拦截 Controller 层抛出的异常，转换为统一返回体格式：

- **BusinessException**：业务异常，提取异常中的 code 和 message 构造 Result 返回；
- **MethodArgumentNotValidException**：JSR-303 参数校验异常，汇总所有字段级别的错误信息，以分号分隔返回；
- **Exception**：兜底异常处理，记录完整堆栈日志，向前端返回 500 错误。

该机制确保所有异常（包括认证鉴权模块抛出的 BusinessException）都能以统一的 JSON 格式返回前端，前端只需关注 Result 中的 code 和 message 字段即可完成统一的错误处理逻辑。

2.12 安全防护措施

认证权限模块在设计中考虑了以下安全防护点：

1. **密码不落盘**：密码仅以 BCrypt 密文形式存储在数据库中，日志中不打印密码参数（AuthService 中密码字段仅参与比对，不输出日志）；
2. **Token 时效性**：JWT Token 有效期 24 小时，过期后必须重新登录，降低 Token 泄露后的安全风险窗口；

3. **接口级权限控制**: 每个需要权限的 API 接口通过 `@RoleRequired` 明确声明角色要求, 防止水平越权 (学生调用教师接口) 和垂直越权 (普通用户调用管理员接口);
4. **注册角色限制**: `AuthService.register()` 方法中硬编码禁止注册 ADMIN 角色, 管理员账号仅由 DBA 在数据库层面手动创建, 防止恶意注册管理员;
5. **账号禁用机制**: `enabled` 字段支持管理员禁用违规或异常账号, 禁用账号在登录时被明确拒绝 (返回 403 禁用提示, 而非模糊的 401 认证失败)。

2.13 认证鉴权全链路时序

用户从登录到访问受保护 API 的完整认证鉴权链路如下:

1. 用户通过 `POST /api/v1/auth/login` 提交用户名和密码;
2. `AuthService` 验证用户名密码, 通过后生成 JWT Token, 写入 Redis, 返回 `LoginResponse` (包含 Token);
3. 前端将 Token 存储在 `localStorage` 中, 并设置 Axios 请求拦截器——每次 HTTP 请求自动在 header 中注入 `Authorization: Bearer <token>`;
4. 用户访问受保护的 API (如 `GET /api/v1/thesis/list`):
 - (a) `JwtFilter` 从 header 提取 Token, 校验签名和过期时间, 将 `userId/username/role` 注入 Request Attribute;
 - (b) `RoleInterceptor` 检查目标 Controller 方法上的 `@RoleRequired` 注解, 校验当前用户角色是否在允许列表中;
 - (c) 鉴权通过后, Controller 方法通过 `request.getAttribute("userId")` 获取当前操作用户 ID, 确保数据隔离 (学生只能操作自己的论文, 教师只能查看自己学院的论文等)。
5. 用户登出时调用 `POST /api/v1/auth/logout`, 服务端删除 Redis 中的 Token 记录, 前端清空 `localStorage` 中的 Token。

该链路的设计确保了从认证到鉴权的完整闭环, 每个环节都有明确的职责边界和异常处理策略, 构成了系统安全体系的基石。